

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ	2
ОПРЕДЕЛЕНИЯ И АББРЕВИАТУРЫ	3
1 ОБЩЕЕ ОПИСАНИЕ	4
1.1 Описание продукта	4
1.2 Функционал продукта.....	4
1.3 Характеристики пользователей	5
2 ДЕТАЛЬНЫЕ ТРЕБОВАНИЯ	6
2.1 Сравнительный анализ аналогичных решений.....	6
2.2 Требования по категориям иерархии Вигерса.....	7
2.3 Диаграмма User Story Map	9
3 ДИАГРАММЫ UML	10
3.1 Диаграмма вариантов использования (Use Case).....	10
3.2 Диаграмма классов (Class)	11
3.3 Диаграмма последовательности (Sequence)	11
4 ПРИМЕНЕНИЕ ПАТТЕРНОВ ПРОЕКТИРОВАНИЯ	13
4.1 Паттерн «Фасад»	13
4.2 Паттерн «Композиция».....	14
4.3 Принцип единственной ответственности	14
4.4 Инкапсуляция	15
4.5 Вывод	16

ВВЕДЕНИЕ

В современных сетевых системах важную роль играет оптимизация передачи данных и снижение нагрузки на внешние ресурсы. При частом обращении к одним и тем же веб-ресурсам возникает необходимость ускорения доступа и фильтрации сетевого трафика. Одним из решений данной проблемы является использование прокси-сервера, поддерживающего локальное сохранение полученных ответов сервера и повторное использование их при последующих идентичных запросах. Это значительно снижает нагрузку на сеть, повышает производительность и безопасность.

Цель работы: разработка HTTP прокси-сервера с кэшированием, фильтрацией по URL-адресам, обеспечивающего ускорение доступа к веб-ресурсам и контроль передаваемого сетевого трафика.

Область применения: система представляет собой программный компонент, реализованный на языке программирования C++ и работающий в локальной сети. Она предназначена для приёма и обработки HTTP-ресурсов, локального сохранения ответов и фильтрации доступа к определённым веб-ресурсам.

ОПРЕДЕЛЕНИЯ И АББРЕВИАТУРЫ

- **Прокси-сервер** — промежуточный сервер, принимающий запросы от клиентов и перенаправляющий их на целевые серверы с фильтрацией по URL-адресам при необходимости;
- **Кэширование** — процесс сохранения результатов ответов серверов локально для повторного использования;
- **TTL (Time To Live)** — время жизни записи в локальном кэше;
- **Чёрный список** — набор URL-адресов и доменных имён, доступ к которым запрещён;
- **Cache HIT** — «попадание» запроса в локальный кэш (данные найдены в кэше);
- **Cache MISS** — «промах» запроса в локальном кэше (данные не найдены в кэше);
- **UML (Unified Modeling Language)** — язык графического описания программных систем;
- **TTFB (Time To First Byte)** — время от начала отправки запроса до получения первого байта ответа от сервера.

1 ОБЩЕЕ ОПИСАНИЕ

Прокси-серверы широко применяются в интернет-архитектуре для оптимизации сетевого взаимодействия, повышения пропускной способности и производительности сети, а также для обеспечения безопасности. Использование локального кэширования позволяет значительно сократить Time To First Byte (TTFB). Разрабатываемая система ориентирована на использование в локальных сетях, где требуется ускорение доступа к часто используемым ресурсам и контроль пользовательской активности.

1.1 Описание продукта

Разрабатываемый продукт представляет собой программную систему, которая принимает HTTP-трафик внутри сети, выполняет его анализ и либо возвращает локальный кэшированный ответ, либо отправляет запрос на целевой сервер. Помимо этого, система поддерживает фильтрацию запросов по URL, что позволяет ограничивать доступ к определённым веб-ресурсам. Все запросы и их результаты фиксируются в журнале аудита системы. Модуль кэширования ответов серверов хранит их с учётом времени жизни (TTL), после истечения которого данные считаются устаревшими и автоматически удаляются.

1.2 Функционал продукта

Разрабатываемая система реализует набор функций, обеспечивающих обработку, оптимизацию и контроль трафика в сети. Ключевой функцией системы является перехват HTTP-запросов от клиентов и их последующая обработка. Сервер принимает запрос, анализирует его и принимает решение о дальнейшем шаге – либо блокировки запроса, либо продолжения его выполнения (т.е. проверки кэша и, в случае необходимости, отправки оригинального запроса на конечный сервер).

Как было сказано ранее, система выполняет кэширование ответов от целевых (конечных) серверов. При повторном обращении к одному и тому же ресурсу в короткий промежуток времени данные будут получены из локального

хранилища без повторного обращения к конечному серверу, что позволяет существенно сократить время отклика (TTFB) и снизить сетевую нагрузку. Перед отправкой запроса осуществляется проверка наличия локальной записи об обращении к серверу в хранилище. В случае присутствия записи (cache hit), она отправляется пользователю. В ином случае (cache miss) запрос перенаправляется на целевой сервер, а его ответ сохраняется локально.

Дополнительно реализован функционал фильтрации по URL-адресам. При получении запроса система анализирует полученный адрес и проверяет его наличие в локальном чёрном списке. При обнаружении запроса на запрещённый веб-ресурс, пользователю возвращается ответ с кодом HTTP-ошибки (403 Forbidden), а факт обращения фиксируется в локальном журнале. Система осуществляет запись в него всех запросов и результатов их обработки (тип запроса, конечный адрес, результат обработки (cache hit/miss) и факт блокировки). Система поддерживает механизм обработки нескольких запросов одновременно, обеспечивая стабильную работу в условиях многопользовательской нагрузки.

1.3 Характеристики пользователей

Система предназначена для нескольких категорий конечных пользователей и реализует различный функционал для каждой из них:

- **Сетевые администраторы:** управляют настройками системы, контролируют доступ к веб-ресурсам и анализируют журналы;
- **Пользователи сети:** отправляют запросы и получают ответы через реализуемый HTTP прокси-сервер;
- **Разработчики и DevOps-инженеры:** используют систему для оптимизации сетевых взаимодействий и обнаружения ограничивающих факторов;

Система не требует дополнительного специального обучения и работает прозрачно для конечного пользователя (применён подход Plug & Play).

2 ДЕТАЛЬНЫЕ ТРЕБОВАНИЯ

Процесс проектирования основан на иерархии требований по Карлу Вигерсу, включающей бизнес-, пользовательские, функциональные и нефункциональные требования. Методология сбора требований к реализуемой системе включала в себя анализ аналогичных решений и формирование пользовательских сценариев (User Stories), что позволило определить ключевые функции системы.

2.1 Сравнительный анализ аналогичных решений

Для уточнения требований были рассмотрены решения, которые решают близкие задачи: проксирование HTTP-трафика, кэширование ответов, фильтрация запросов и журналирование событий. Анализ выполнен по критериям, значимым для данного проекта: наличие кэширования, поддержка фильтрации URL, сложность развёртывания, возможность расширения и применимость в локальной сети.

Таблица 2.1 — Сравнительный анализ аналогичных решений

Решение	Назначение и подход	Сильные стороны	Ограничения для проекта
Squid	Полноценный кэширующий прокси-сервер для сетевой инфраструктуры.	Развитые механизмы кэширования, списки доступа, журналирование, высокая производительность.	Сложная настройка и избыточность для учебной реализации; логика работы скрыта внутри готового продукта.
Nginx cache	Веб-сервер и обратный прокси с модулем кэширования HTTP-ответов.	Высокая скорость, устойчивость под нагрузкой, гибкая конфигурация кэша.	Ориентирован прежде всего на обратное проксирование;

			фильтрация пользовательских запросов требует дополнительных правил и модулей.
Apache Traffic Server	Производительный прокси и кэширующий сервер для крупных сетей.	Масштабируемость, развитый кэш, поддержка сложных сценариев проксирования.	Сложная архитектура и высокий порог входа; нецелесообразен для компактного учебного приложения.

По результатам анализа выбрана компактная архитектура собственного HTTP прокси-сервера. В отличие от промышленных решений, система не должна заменять готовую корпоративную платформу, а должна наглядно реализовать ключевые механизмы: приём запроса, проверку URL, поиск ответа в кэше, обращение к целевому серверу при промахе кэша и журналирование результата.

2.2 Требования по категориям иерархии Вигерса

Таблица 2.2 — Требования по категориям иерархии Вигерса

Категория	Требование	Критерий проверки
Бизнес-требование	Система должна эффективно снижать количество повторных обращений к целевым серверам за счёт кэширования HTTP-ответов.	При повторном запросе к одному URL ответ возвращается из локального кэша, если его TTL не истёк.

Пользовательское	Пользователь должен иметь возможность отправлять HTTP-запросы через прокси-сервер.	Запрос клиента принимается прокси-сервером и обрабатывается системой.
Пользовательское	При обращении к заблокированному URL пользователь должен получить сообщение об отказе.	Система возвращает HTTP-ответ со статусом 403 Forbidden.
Функциональное	Система должна принимать HTTP-запросы от клиентов.	Прокси-сервер принимает входящий HTTP-запрос на заданном порту.
Функциональное	Система должна проверять URL запроса по чёрному списку.	Если полученный URL в чёрном списке, запрос должен быть заблокирован с соответствующей страницей для пользователя.
Функциональное	Система должна возвращать ответ из кэша при наличии актуальной записи.	При попадании в кэш (Cache Hit) клиент получает сохранённый ответ без обращения к целевому серверу.
Функциональное	При отсутствии актуальной записи в кэше, система должна	При отсутствии кэша (Cache Miss) прокси получает ответ от

	обратиться к целевому серверу.	целевого сервера и сохраняет его в кэш.
Функциональное	Система должна вести журнал обработки запросов.	В журнал записываются URL, результат обработки и состояния кэша, факт блокировки.
Нефункциональное	Система должна обеспечивать конкурентную обработку клиентских запросов, не блокируя выполнение других задач.	При одновременной отправке нескольких запросов сервер должен возвращать корректные ответы каждому клиенту.
Нефункциональное	Код системы должен быть модульным, поддерживать расширение и сопровождение.	Кэширование, фильтрация и журналирование реализованы отдельными классами.
Нефункциональное	Система должна поддерживать настройку времени жизни записей кэша.	TTL задаётся в настройках и учитывается при проверке актуальности записи.

2.3 Диаграмма User Story Map

Для структурирования пользовательских требований и определения приоритетов разработки была реализована карта пользовательских историй (User Story Map). Диаграмма отражает основные сценарии взаимодействия пользователей с системой, а также распределение функционала по приоритетам.

Помимо этого, диаграмма имеет разделение пользователей на роли: пользователи и администратор (каждая имеет собственные сценарии использования системы).

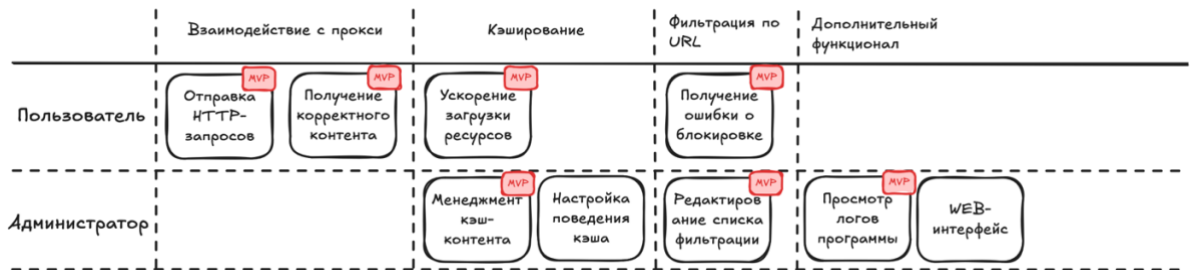


Рисунок 2.1 — Диаграмма User Story Map

3 ДИАГРАММЫ UML

Для описания реализуемой системы используются UML-диаграммы, отражающие различные аспекты её функционирования и пользовательский опыт использования.

3.1 Диаграмма вариантов использования (Use Case)

Данная диаграмма отображает взаимодействие конечных пользователей с системой.

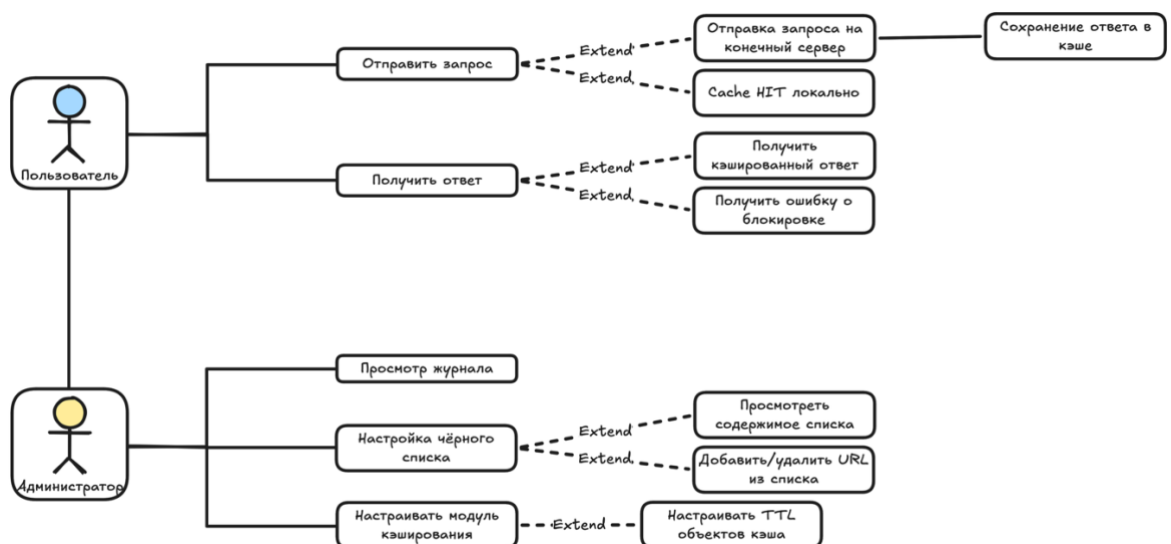


Рисунок 3.1 — Диаграмма вариантов использования

Пользователь взаимодействует с реализуемой системой посредством отправки HTTP-запросов и получения ответов от сервера. Администратор может управлять чёрным списком веб-ресурсов и анализировать журнал запросов.

3.2 Диаграмма классов (Class)

Данная диаграмма показывает связи и взаимодействия между множеством классов программы, отражает структуру системы и основные компоненты.

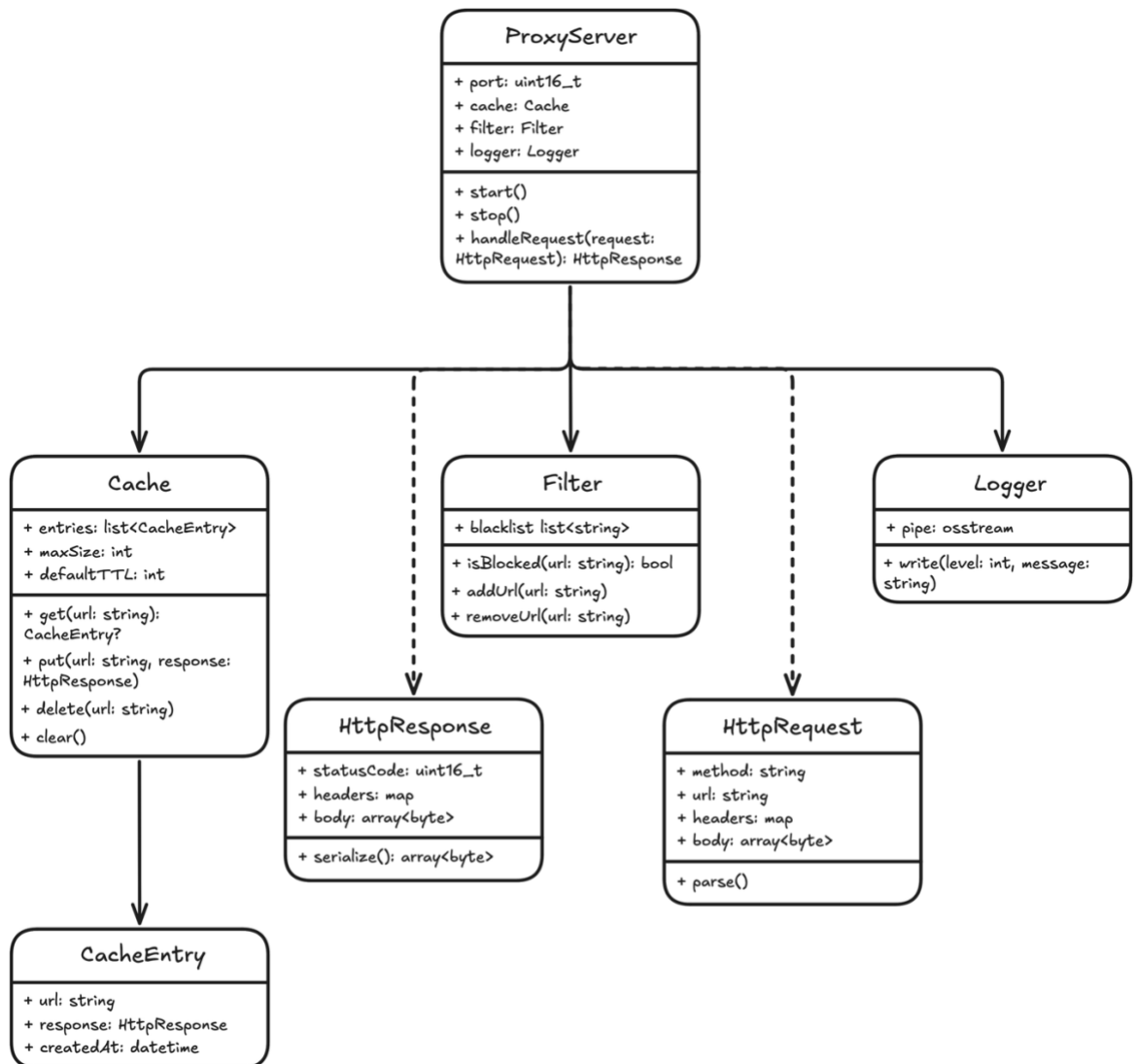


Рисунок 3.2 — Диаграмма классов

3.3 Диаграмма последовательности (Sequence)

Диаграмма последовательности описывает процесс обработки запроса продуктом.

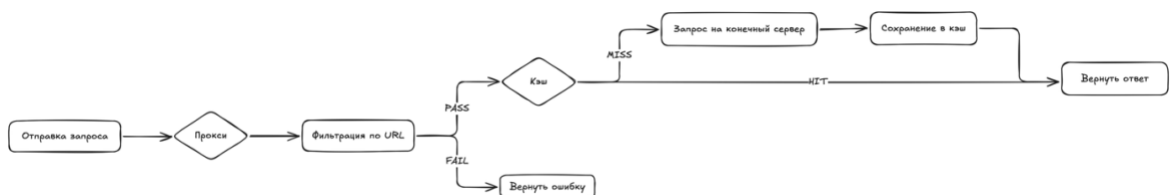


Рисунок 3.3 — Диаграмма алгоритма обработки запроса

Пользователь отправляет HTTP-запрос прокси-серверу. Получив запрос, он проверяет, не находится ли веб-ресурс в чёрном списке. После этого сервер проверяет наличие данных для запроса в локальном хранилище (кэше). При наличии актуального ответа, он возвращается немедленно. Если данные отсутствуют, прокси отправляет запрос на целевой сервер, получает ответ, сохраняет его в локальном хранилище и отправляет пользователю.

4 ПРИМЕНЕНИЕ ПАТТЕРНОВ И ПРИНЦИПОВ ПРОЕКТИРОВАНИЯ

При проектировании HTTP прокси-сервера были использованы паттерны и принципы проектирования, обеспечивающие модульность, расширяемость, читаемость и надёжность реализуемого продукта. Их применение позволяет уменьшить связанность между компонентами, упростить сопровождение кода и обеспечить соответствие нефункциональным требованиям, связанным с производительностью, расширяемостью и использованием объектно-ориентированного подхода.

4.1 Паттерн «Фасад»

В реализуемой системе роль фасада выполняет класс `ProxyServer`. Для внешнего клиента он предоставляет простой и ограниченный интерфейс, состоящий из методов `start()`, `stop()` и `handleRequest()`. При этом клиентский код не взаимодействует напрямую с подсистемами `Cache`, `Filter` и `Logger`, а обращается только к объекту прокси-сервера. Использование паттерна «Фасад» позволяет скрыть внутреннюю сложность системы.

Например, при вызове метода `handleRequest()` объект `ProxyServer` последовательно координирует работу нескольких подсистем. Сначала он передаёт URL-адрес в модуль фильтрации и вызывает метод `Filter::isBlocked()`. Если адрес находится в чёрном списке, формируется объект `HttpResponse` со статус-кодом «403 Forbidden», а информация о блокировке передаётся в журнал через `Logger::write()`. Если ресурс не заблокирован, сервер обращается к модулю кэширования и вызывает `Cache::get()`. При наличии актуальной записи ответ сразу возвращается клиенту. Если запись отсутствует, прокси-сервер получает ответ от целевого сервера, сохраняет его в кэше через `Cache::put()` и только после этого отправляет клиенту.

Таким образом, фасад упрощает использование системы, так как скрывает детали внутренней логики и предоставляет единый вход для работы с подсистемами. Это также облегчает развитие проекта: внутреннюю реализацию модулей можно изменять без изменения внешнего интерфейса.

4.2 Композиция объектов

Архитектура системы построена с использованием композиции. Класс `ProxyServer` содержит внутри себя объекты `Cache`, `Filter` и `Logger`, то есть между ними реализована связь типа «has-a». Это означает, что данные подсистемы являются составными частями прокси-сервера и не рассматриваются как полностью самостоятельные объекты в рамках внешнего взаимодействия. Применение композиции делает структуру программы более логичной и устойчивой.

Жизненный цикл подсистем напрямую связан с жизненным циклом владельца. При создании объекта `ProxyServer` автоматически создаются его внутренние модули, а при уничтожении прокси-сервера они также корректно освобождаются. Это снижает вероятность ошибок управления памятью и упрощает контроль над внутренним состоянием системы.

Кроме того, композиция используется и на более низком уровне модели. Например, класс `Cache` хранит набор объектов, каждый из которых представляет отдельную запись локального кэша. За счёт этого структура данных остаётся естественной и хорошо соответствует предметной области.

4.3 Принцип единственной ответственности

В системе соблюдается принцип единственной ответственности согласно которому каждый класс должен отвечать только за одну логически завершённую задачу.

Класс `Cache` отвечает исключительно за хранение, поиск и удаление кэшированных ответов. Класс `Filter` управляет чёрным списком URL-адресов и выполняет проверку допустимости обращения к ресурсу. Класс `Logger` отвечает

за фиксацию событий и результатов обработки запросов. Классы `HttpRequest` и `HttpResponse` реализуют HTTP-сообщения и обеспечивают парсинг и сериализацию. Класс `ProxyServer` не реализует самостоятельно кэширование, фильтрацию или журналирование, а лишь координирует работу соответствующих модулей.

Такое разделение обязанностей делает код более понятным и поддерживаемым. Изменение логики фильтрации не требует изменения механизма кэширования, а доработка формата логов не влияет на обработку HTTP-запросов. Благодаря этому система легче расширяется и лучше подходит для модульного тестирования.

4.4 Инкапсуляция

Важным принципом, использованным при проектировании системы, является инкапсуляция. Внутреннее состояние объектов скрыто от внешнего кода и доступно только через публичные методы.

Например, класс `Cache` скрывает внутреннюю коллекцию кэшированных ответов, параметры размера и времени жизни записей, предоставляя вместо прямого доступа методы `get()`, `put()`, `delete()` и `clear()`. Класс `Filter` не позволяет внешнему коду напрямую изменять список запрещённых адресов, а предоставляет для этого методы `addUrl()`, `removeUrl()` и `isBlocked()`. Класс `Logger` также инкапсулирует внутренний механизм вывода и предоставляет единый метод `write()` для записи сообщений, вне зависимости от финального хранилища сообщений (к примеру, файл или консоль вывода).

Инкапсуляция защищает внутренние инварианты объектов и предотвращает неконтролируемое изменение их состояния. Кроме того, она позволяет изменять внутреннюю реализацию классов без влияния на клиентский код. Это особенно важно для долгосрочного развития проекта, так как упрощает рефакторинг и замену внутренних структур данных.

4.5 Вывод

Подводя итог, можно сказать, что при разработке HTTP-прокси сервера были применены как классические паттерны проектирования, так и фундаментальные принципы объекто-ориентированного программирования. Совокупное использование этих решений делает систему более понятной, устойчивой к изменениям и удобной для дальнейшего развития.